

透過 Gemini Enterprise Agent Platform 的 Agent Gateway 管理代理工作負載

來源：<https://codelabs.developers.google.com/cloudnet-agent-gateway?hl=zh-tw>

步驟數：19

在 Agent Runtime 部署受控的多工具 ADK 代理，透過 Agent Gateway 呼叫 Cloud Run 上的 MCP 伺服器。

目錄

1. 簡介
2. 事前準備
3. 複製存放區
4. 建立 Terraform 狀態 bucket 和後端設定
5. (選用) 建立公開 Cloud DNS 區域
6. 設定 Terraform 變數
7. 使用 Terraform 部署基礎架構
8. 檢查 Agent Registry 端點
9. 檢查代理閘道設定
10. 檢查 IAP 和 Model Armor 授權
11. 在 Cloud Run 中建構及部署 MCP 伺服器
12. 將房貸代理部署至 Agent Runtime
13. 授予代理每個 MCP 伺服器的輸出權限
14. 在 Agent Platform 控制台測試代理程式
15. 強制執行 IAP 授權
16. 設定及測試 Gemini Enterprise
17. 疑難排解與常見修正方式
18. 清除所用資源
19. 恭喜

1. 簡介

不公開預先發布版 - **Agent Gateway** 如要要求存取權，以便透過 Agent Runtime 使用 Agent Gateway，請參閱[存取權要求頁面](#)。

Gemini Enterprise Agent Platform 是開放式平台，可協助您建構、擴充、管理及最佳化調整以自家資料為基準的企業級 AI 代理。

Agent Runtime 提供代管的執行環境，可在 Google Cloud 中安全地執行代理，例如使用開放原始碼 **Agent Development Kit (ADK)** 建構的代理。

本程式碼研究室將探討如何使用這些核心建構區塊，控管 Gemini Enterprise 中由使用者啟動的代理，確保代理安全地存取內部工具。

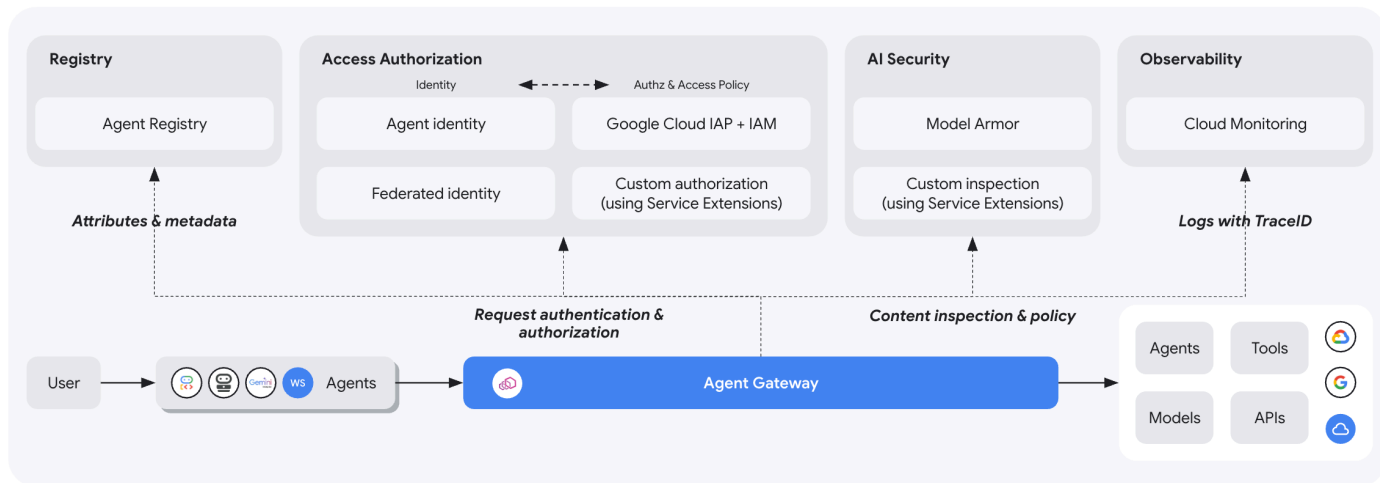
關於 Agent Gateway

Agent Gateway 是平台代理管理套件的網路元件，做為所有代理程式互動的網路進入和退出點，可讓安全管理員強制執行集中式控管，開發人員不必管理複雜的網路基本元素。

這項服務可協助您完成兩項主要受控存取路徑：

- **用戶端到代理程式 (傳入)：**確保外部用戶端 (例如 Cursor 或 Gemini CLI) 與代理程式之間的通訊安全。
- **代理程式到任何位置 (輸出)：**保護在 Google Cloud 上執行的代理程式與伺服器、工具或 API 之間的通訊安全。

在本程式碼研究室中，您將著重於「代理至任意目的地 (輸出)」模式。



為強制執行[安全政策](#)，Agent Gateway 會與生態系統的其餘部分緊密整合：

- **Agent Registry**：核准的代理和工具 (包括第三方 MCP 伺服器) 中央程式庫。
- **代理身分**：每個代理都有專屬的可追蹤身分，並自動以端對端 mTLS 保護。
- **Identity-Aware Proxy (IAP) 和 IAM**：預設強制執行層，會根據精細的 IAM 權限驗證代理程式的身分，然後允許呼叫特定工具。
- **Model Armor**：透過 Service Extensions 整合的 AI 安全防護機制，可清除內容並防範提示詞注入攻擊或資料外洩。

部署模式 (Cloud Run 的公有與私有網路)

為方便使用本程式碼研究室，您可以為部署在 **Cloud Run** 的內部工具 (MCP 伺服器) 選擇以下兩種網路路徑：

1. **預設 (公開連入)**：MCP 伺服器會部署到 Cloud Run，並使用公開主機名稱 (`ingress=all`)。流量會透過標準 `*.run.app` URL 從代理程式路由至工具。這不需要自訂 DNS 網域，是瞭解控管概念最快的方法。
2. **安全 (私人網路)**：完全私密的選用架構。MCP 伺服器受到限制 (`ingress=internal-and-cloud-load-balancing`)，並透過無伺服器 NEG 的內部應用程式負載平衡器公開。您必須擁有公開 DNS 網域，才能佈建 Google 代管的憑證。

設定 Terraform 時，您會選取偏好的路徑。

如要進一步瞭解 Cloud Run 的網路端點輸入，請參閱[說明文件](#)。

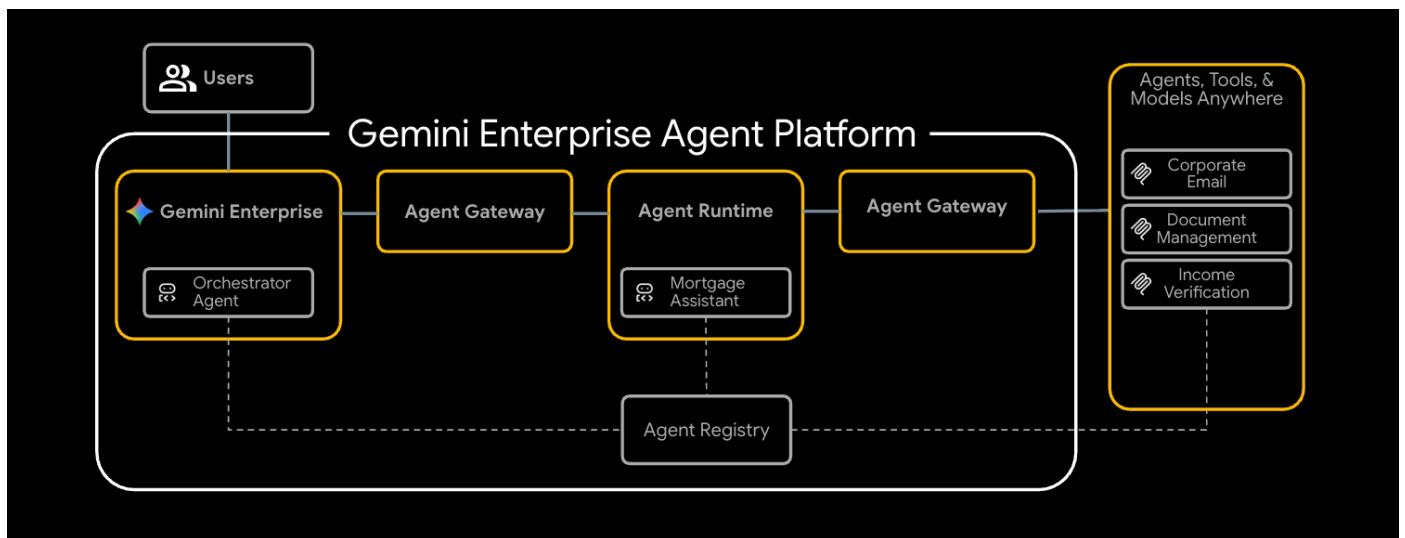
學習內容

- 使用 Terraform 佈建核心基礎架構堆疊
- 在 Cloud Run 建構及部署內部工具做為 MCP 伺服器
- 使用 PSC 介面輸出，將 ADK 代理部署至 Agent Runtime
- 設定 Agent Gateway 服務擴充功能，以根據身分 (IAM) 和內容篩選 (Model Armor) 管理存取權
- 追蹤並驗證代理程式的端對端安全執行作業

軟硬體需求

- 網路瀏覽器，例如 [Chrome](#)
- 已啟用計費功能的 Google Cloud 專案，且您擁有擁有者存取權
- 機構層級的 IAM 權限 (本程式碼研究室會授予機構範圍的角色)
- 您控管的網域已委派給 Cloud DNS (適用於公開代管憑證)
- 熟悉 Terraform、`gcloud` 和 Google Cloud 網路基礎知識

程式碼研究室拓撲



在本程式碼研究室中，您將部署端對端抵押貸款承銷代理程式，並與三種內部工具安全地通訊。

首先，您將佈建基礎網路，包括虛擬私有雲和設定為 Agent Gateway 的內部應用程式負載平衡器。接下來，請將三個 Model Context Protocol (MCP) 伺服器部署至 Cloud Run。這些工具可做為內部專屬工具：

- 文件管理 (`legacy-dms`)
- 公司電子郵件地址 (`corporate-email`)
- 收入驗證 (`income-verification`)

準備好工具後，您會將使用 ADK 建構的房貸助理 (`mortgage-agent`) 部署至 Agent Runtime。您將設定這個代理，透過 PSC 介面進行私人輸出，並透過 Agent Registry 啟用執行階段工具探索功能。

如要確保流程安全無虞，請使用兩個服務擴充功能設定 Agent Gateway。首先，`REQUEST_AUTHZ` 擴充功能會根據各項工具的 IAM 政策驗證代理程式身分，確保代理程式只存取授權工具。其次，使用 Model Armor 的 `CONTENT_AUTHZ` 擴充功能會篩選代理程式的提示和回覆。

最後，您會在 Gemini Enterprise 中註冊代理程式、以一般使用者身分觸發抵押貸款核保工作，並使用 Cloud Trace 驗證安全受控的執行作業。

本程式碼研究室適合各種程度的平台和安全工程師。預計完成這項作業需要約 **100 分鐘**。

步驟 2 · 約 5 分鐘

2. 事前準備

建立專案並進行驗證

建立新的 GCP 專案 (或重複使用現有專案)，並啟用帳單功能，然後驗證 Cloud Shell 或本機電腦：

```
gcloud auth login
gcloud auth application-default login
gcloud config set project <your-project-id>
```

啟用啟動程序 API

Terraform 的基礎模組會在首次套用時啟用約 30 個 API，但 `terraform init` 和 GCS 狀態值區塊需要一小組啟動設定：

```
gcloud services enable \
  compute.googleapis.com \
  serviceusage.googleapis.com \
  cloudresourcemanager.googleapis.com \
  iam.googleapis.com \
  storage.googleapis.com \
  dns.googleapis.com
```

安裝必要工具

安裝工具鍊。Cloud Shell 中已提供大部分的這些項目；工作站則需：

```
# uv (Python package manager)
curl -LsSf https://astral.sh/uv/install.sh | sh

# skaffold
curl -Lo skaffold https://storage.googleapis.com/skaffold/releases/latest/skaffold-linux-
amd64 && \
  sudo install skaffold /usr/local/bin/

# envsubst (gettext)
sudo apt-get install -y gettext-base
```

您也需要 **Terraform >= 1.12.2**、**Python 3.12** 以上版本，以及 **Google Cloud SDK** (`gcloud`)。

設定環境變數

本程式碼研究室的其餘部分會假設這些項目已匯出至殼層。

如果您要執行選用的安全網路路徑，請定義 `DOMAIN_NAME` (例如 `agw.example.com`)；否則可以忽略此步驟。

```
export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format='value(projectNumber)')
export ORG_ID=$(gcloud projects get-ancestors $PROJECT_ID | awk '$2 == "organization" {print $1}')
export REGION="us-central1"

# Only required if using the secure private networking path
export DOMAIN_NAME="agw.example.com"
```

確認所有變數都已正確填入，您應該會看到三個傳回值。

```
echo $PROJECT_ID
echo $PROJECT_NUMBER
echo $ORG_ID
```

如果系統未自動填入機構 ID，請手動尋找並設定。

```
gcloud organizations list
export ORG_ID=ID_FROM_OUTPUT
```

步驟 3 · 約 1 分鐘

3. 複製存放區

```
git clone https://github.com/GoogleCloudPlatform/cloud-networking-solutions.git
cd cloud-networking-solutions
cd demos/agent-gateway
```

快速瀏覽示範目錄中的內容：

src/	MCP servers (legacy-dms, corporate-email, income-verification-api) +
mortgage-agent	
terraform/	Root Terraform config + modules (foundation, networking, agent-gateway,
model-armor, ...)	
cloudrun/	Cloud Run service definitions (rendered from .yaml.tpl via envsubst)
scripts/	grant_agent_mcp_egress.sh – per-MCP IAP egressor binding
skaffold.yaml.tpl	Skaffold pipeline that builds + deploys all three MCP services to Cloud
Run	

步驟 4 · 約 2 分鐘

4. 建立 Terraform 狀態 bucket 和後端設定

建立 GCS bucket 來保存遠端狀態，然後複製後端範本：

```
gcloud storage buckets create gs://${PROJECT_ID}-tfstate \  
  --location=${REGION} \  
  --uniform-bucket-level-access  
  
cp terraform/example.backend.conf terraform/backend.conf
```

使用您的值編輯 `terraform/backend.conf`：

```
bucket = "<your-project-id>-tfstate"  
prefix = "agent-gateway"
```

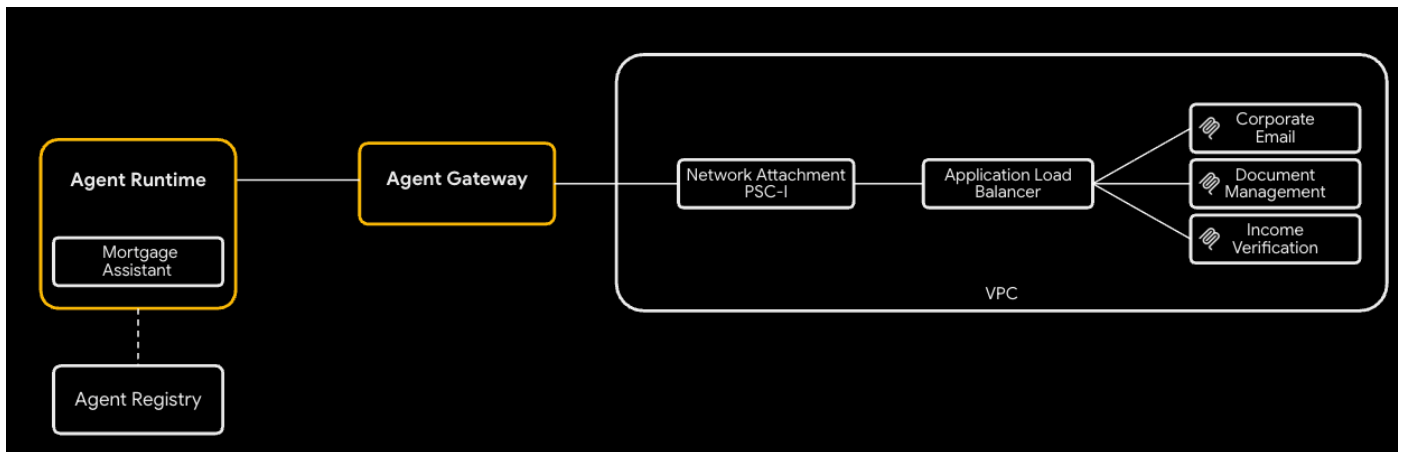
`terraform/backend.conf` 會遭到 Git 忽略。每位使用者都會維護自己的副本。

5. (選用) 建立公開 Cloud DNS 區域

如果使用預設路徑，請略過這個步驟。如要使用最簡單的設定和公開 Cloud Run 網址，請直接前往下一個步驟。

根據預設，本實驗室的 Cloud Run [輸入設定](#)會設為 `all`，且代理程式登錄檔會在公開 `*.run.app` 網址註冊每個 MCP 伺服器，因此不需要額外的 DNS、憑證或負載平衡器。如要改用私人網路 (Cloud Run 搭配內部應用程式負載平衡器後方的 `ingress = internal-and-cloud-load-balancing`)，您也需要公開 Cloud DNS 區域，Certificate Manager 才能驗證負載平衡器憑證。

私人網路的高階流程



如要使用私人網路方法，請按照下列步驟操作：

1. 建立公開 Cloud DNS 區域 - Certificate Manager 會將 CNAME 寫入該區域，藉此驗證區域代管憑證：

```
gcloud dns managed-zones create agw-example-com \
  --dns-name="${DOMAIN_NAME}." \
  --description="Public zone for ${DOMAIN_NAME}" \
  --visibility=public
```

Terraform 會自動建立 `mcp.${DOMAIN_NAME}` 的對應私人區域 (由 MCP 內部 LB 和 Agent Runtime 的 DNS 對等互連使用)，您不需要手動建立。如果關閉私人網路，系統就不會佈建公開或私人區域。

如要深入瞭解本示範使用的公開和私人水平分割 DNS 模式 (代管區域設定、跨 VPC 的 DNS 對等互連、憑證驗證流程)，請參閱 [DNS 設定指南](#)。

步驟 6 · 約 5 分鐘

6. 設定 Terraform 變數

複製範例 tfvars 並加以編輯：

```
cp terraform/example.tfvars terraform/terraform.tfvars
```

有兩個受 `enable_cloud_run_private_networking` 限制的試用路徑。

預設路徑：Cloud Run，具有公開輸入功能

最簡單的設定：針對預設路徑，您只需要在 `terraform.tfvars` 中編輯三個值，檔案中的其他變數都已設有適合用於示範的預設值。

```
# GCP project ID where all resources will be created.
project_id = "my-gcp-project-id"

# GCP organization ID (numeric).
organization_id = "123456789012"

# Members granted demo-wide roles
platform_admin_members = ["user:admin@example.com"]

# IAP Enforcement Mode ("DRY_RUN" or null)
agent_gateway_iap_iam_enforcement_mode = "DRY_RUN"
```

我們將在「僅限稽核」模式下模擬測試 IAP，測試 IAP 授權政策，但不強制執行。驗證部署作業後，我們會將強制執行模式更新為 `null`，以強制執行政策。

私人網路 (選填)

設定 `enable_cloud_run_private_networking = true` 並新增下列變數，即可佈建完整的安全堆疊：

- 內部應用程式負載平衡器
- Google 代管的憑證
- Cloud Run (搭配 `ingress = internal-and-cloud-load-balancing`)
- 代理閘道 DNS 對等互連。

```

enable_cloud_run_private_networking = true

# DNS — must end with a trailing dot, must match a Cloud DNS zone you own
dns_zone_domain                    = "agw.example.com."
enable_certificate_manager = true

# mcp_internal_dns_zone.domain MUST be a real subdomain of dns_zone_domain so
# Certificate Manager can issue a Google-managed cert.
mcp_internal_dns_zone = {
  name     = "mcp-server-internal"
  domain   = "mcp.agw.example.com."
}

# Must match mcp_internal_dns_zone.domain so Agent Engine resolves MCP
# hostnames over the PSC interface peering.
psc_interface_dns_zone = {
  name     = "mcp-server-internal"
  domain   = "mcp.agw.example.com."
}

mcp_lb_protocol = "HTTPS"

```

上方的 Cloud Run 圖片指向 Google 預留位置。這是刻意設計的行為，因為 Terraform 會使用 `lifecycle { ignore_changes = [template[0].containers[0].image] }` 建立 Cloud Run 服務，因此後續的 `scaffold run` 叫用會取代映像檔，而 Terraform 不會反制。

`agent_gateway_subnet_cidr` 必須至少為 `/28`、RFC 1918，且不得與 `10.0.0.0/24`、`10.0.1.0/24` 或 `10.0.2.0/24` 重疊。這些範圍是預留給 Agent Gateway 輸出使用。

MCP 內部負載平衡器會使用相同的 Google 管理區域憑證，為 `*.mcp.${DOMAIN_NAME}` 提供 TLS 服務。請勿在 `enable_agent_gateway = true` 和 `enable_cloud_run_private_networking = true` 時，將 `mcp_internal_dns_zone.domain` 設為無法路由的尾碼，例如 `mcp-server.internal.`。

Agent Gateway 目前不會驗證自行簽署的回退憑證。Terraform 設定會在規劃時強制執行這項操作 (私人網路關閉時會略過，因為沒有要驗證的 LB 憑證)。 `check`

7. 使用 Terraform 部署基礎架構

初始化、檢查及套用：

```
cd terraform
terraform init -backend-config=backend.conf
terraform plan -out=tfplan
terraform apply tfplan
```

`terraform apply` 會在預設路徑上佈建約 40 個資源，並在全新專案上花費 8 到 10 分鐘
(`enable_cloud_run_private_networking = true` 時約 60 個資源 / 15 到 20 分鐘)。這會建立：

- 專案基礎 (API、服務身分、配額)
- 虛擬私有雲、子網路 (主要、僅限 Proxy、PSC、PSC 介面、代理程式閘道共置)、Cloud NAT、防火牆規則
- Cloud Run 映像檔的 Artifact Registry 存放區
- 三個 Cloud Run 服務 + 每個服務的執行階段服務帳戶 (根據預設，輸入 = `all`；私人網路開啟時為 `internal-and-cloud-load-balancing`)
- Model Armor 範本 + IAM
- Agent Gateway、PSC-I 網路連結、IAP 和 Model Armor 擴充功能、授權政策，以及專案層級的 `roles/iap.egressor` 授權
- Agent Registry 端點 (Vertex AI、IAP、Discovery Engine 等)，以及三個 MCP 伺服器 (預設註冊於 `*.run.app/mcp`；私人網路開啟時註冊於 `./mcp`)

僅限於 `enable_cloud_run_private_networking = true`：

- 內部區域應用程式負載平衡器搭配無伺服器 NEG (網址遮罩路由) + 私人 DNS A 記錄
 - 附加至 VPC 的 MCP 私人 DNS 區域 (`mcp..`)
 - 公開 DNS 區域模組 (Certificate Manager DNS 授權) + 區域 Google 管理的憑證
 - PSC 介面 DNS 區域 (如果沒有要解析的私人主機名稱，就會成為孤立區域，因此也會受到主旗標的限制)
 - `mcp..` 的 Agent Gateway DNS 對接 (自動預先附加)
-

8. 檢查 Agent Registry 端點

僅供參考。Terraform 的 `agent-registry-endpoints` 模組已為您執行本節中的指令。顯示這些項目是為了讓您瞭解註冊 MCP 可探索端點的確切樣貌。

Agent Registry 是每個專案的服務目錄 (Google API 和您自己的 MCP 伺服器)，代理會在執行階段探索這些服務。抵押貸款代理程式會在啟動時讀取該檔案，並動態繫結工具，代理程式程式碼或部署指令中不會有 MCP 網址。

端點

Terraform 代表您執行的作業：針對 `agent_registry_google_apis` 中的每個 Google API，註冊五種變體 (全域、mTLS 全域、區域、區域 mTLS、區域 REP)。例如，針對 `aiplatform`：

```
gcloud alpha agent-registry services create aiplatform \
  --project=${PROJECT_ID} --location=${REGION} \
  --display-name="Vertex AI Platform" \
  --endpoint-spec-type=no-spec \
  --interfaces="url=https://aiplatform.googleapis.com,protocolBinding=JSONRPC"

gcloud alpha agent-registry services create aiplatform-mtls \
  --project=${PROJECT_ID} --location=${REGION} \
  --display-name="Vertex AI Platform mTLS" \
  --endpoint-spec-type=no-spec \
  --interfaces="url=https://aiplatform.mtls.googleapis.com,protocolBinding=JSONRPC"

gcloud alpha agent-registry services create ${REGION}-aiplatform \
  --project=${PROJECT_ID} --location=${REGION} \
  --display-name="Vertex AI Platform Locational" \
  --endpoint-spec-type=no-spec \
  --interfaces="url=https://${REGION}-aiplatform.googleapis.com,protocolBinding=JSONRPC"

gcloud alpha agent-registry services create aiplatform-${REGION}-rep \
  --project=${PROJECT_ID} --location=${REGION} \
  --display-name="Vertex AI Platform Regional (REP)" \
  --endpoint-spec-type=no-spec \
  --interfaces="url=https://aiplatform.${REGION}.rep.googleapis.com,protocolBinding=JSONRPC"
```

MCP 伺服器

Terraform 也會為您註冊 3 個 MCP 伺服器，如要註冊其他 MCP 伺服器，請按照[文件中的步驟](#)操作。

```
gcloud alpha agent-registry services create legacy-dms \
--project=${PROJECT_ID} \
--location=${REGION} \
--display-name="Legacy DMS" \
--mcp-server-spec-type=tool-spec \
--mcp-server-spec-content=src/legacy-dms/toolspec.json \
--interfaces=url=https://dms.${DOMAIN_NAME}/mcp,protocolBinding=JSONRPC
```

驗證已註冊的端點和 MCP 伺服器。

```
gcloud alpha agent-registry services list \
--project=${PROJECT_ID} --location=${REGION} \
--format="value(displayName,name)"

gcloud alpha agent-registry mcp-servers list \
--project=${PROJECT_ID} --location=${REGION} \
--format="value(displayName,name)"
```

資料來源：`terraform/modules/agent-registry-endpoints/scripts/register_endpoints.sh.tpl`

9. 檢查代理閘道設定

僅供參考。Terraform 已匯入這個代理程式閘道。下列 YAML 是 `terraform/modules/agent-gateway/main.tf` 的對等文字。

Agent Gateway 是由 Google 管理的控管層，介於 Agent Runtime 和您的工具之間。在 `AGENT_TO_ANYWHERE` 模式中，這項服務會繫結至專案的 Agent Registry，並透過客戶擁有的 PSC 介面輸出，因此可以連線至 VPC 中的私人 MCP 伺服器。

如果您是手動匯入這個閘道，YAML 應如下所示：

```
# agent-gateway.yaml - for reference only, Terraform already created this
name: agent-gateway
protocols: [MCP]
googleManaged:
  governedAccessPath: AGENT_TO_ANYWHERE
registries:
  - "//agentregistry.googleapis.com/projects/${PROJECT_ID}/locations/${REGION}"
networkConfig:
  egress:
    networkAttachment: projects/${PROJECT_ID}/regions/${REGION}/networkAttachments/agent-
gateway-na
  dnsPeeringConfig:
    domains:
      - mcp.${DOMAIN_NAME}.
    targetProject: ${PROJECT_ID}
    targetNetwork: projects/${PROJECT_ID}/global/networks/gateway-vpc
```

```
gcloud alpha network-services agent-gateways import agent-gateway \
  --source=agent-gateway.yaml \
  --location=${REGION}
```

確認 Terraform 建立的閘道：

```
gcloud alpha network-services agent-gateways describe agent-gateway \
  --location=${REGION}
```

目前的 `terraform-provider-google-beta` 尚未公開 `network_config.dns_peering_config`。Terraform 會透過 `networkservices.googleapis.com/v1beta1` 上的後套用 `PATCH` 套用此項目。您會在模組的 `terraform_data.dns_peering` 中看到這項資訊。

10. 檢查 IAP 和 Model Armor 授權

僅供參考。Terraform 已建立這兩個擴充功能和這兩項政策。以下指令直接來自 `terraform/modules/agent-gateway/main.tf`。

代理閘道會將授權委派給**服務擴充功能**。以下兩個政策設定檔涵蓋了這個試用版：

- **REQUEST_AUTHZ**：在標頭階段，每個要求都會評估一次。這裡用於呼叫 **IAP**，檢查呼叫代理身分在目標 MCP 伺服器上是否具有 `roles/iap.egressor`。
- **CONTENT_AUTHZ**：將主體事件串流至擴充功能，以進行內容清除。這裡用於呼叫 **Model Armor**，透過 Sensitive Data Protection (SDP) 篩選提示詞注入、越獄、RAI 違規行為和 (選用) PII。

IAP REQUEST_AUTHZ 擴充功能

```
cat > iap-authz-extension.yaml <<EOF
name: agent-gateway-iap-authz
service: iap.googleapis.com
failOpen: true
timeout: 1s
EOF

gcloud beta service-extensions authz-extensions import agent-gateway-iap-authz \
  --source=iap-authz-extension.yaml \
  --location=${REGION} \
  --project=${PROJECT_ID}
```

使用 `REQUEST_AUTHZ` 政策將其繫結至代理程式閘道：

```
curl -fsS -H "Authorization: Bearer $(gcloud auth print-access-token)" \
-H "Content-Type: application/json" \
-X POST
"https://networksecurity.googleapis.com/v1alpha1/projects/${PROJECT_ID}/locations/${REGION}/authzPolicies?authz_policy_id=agent-gateway-iap-policy" \
-d '{
  "name": "agent-gateway-iap-policy",
  "policyProfile": "REQUEST_AUTHZ",
  "action": "CUSTOM",
  "target": {
    "resources": [
      "projects/"'"${PROJECT_ID}"'/locations/"'"${REGION}"'/agentGateways/agent-gateway"
    ]
  },
  "customProvider": {
    "authzExtension": {
      "resources": [
        "projects/"'"${PROJECT_ID}"'/locations/"'"${REGION}"'/authzExtensions/agent-gateway-iap-authz"
      ]
    }
  }
}'
```

Model Armor CONTENT_AUTHZ 擴充功能

擴充功能的 `metadata.model_armor_settings` 會攜帶 Model Armor 用於評估每次呼叫的請求和回應範本 ID：

```
cat > ma-extension.yaml <<EOF
name: agent-gateway-ma-authz
service: modelarmor.${REGION}.rep.googleapis.com
failOpen: true
timeout: 1s
metadata:
  model_armor_settings: '[
    {
      "request_template_id": "projects/${PROJECT_ID}/locations/${REGION}/templates/agw-request-template",
      "response_template_id": "projects/${PROJECT_ID}/locations/${REGION}/templates/agw-response-template"
    }
  ]'
```

```
EOF

gcloud beta service-extensions authz-extensions import agent-gateway-ma-authz \
--source=ma-extension.yaml \
--location=${REGION} \
--project=${PROJECT_ID}
```

```
curl -fsS -H "Authorization: Bearer $(gcloud auth print-access-token)" \
-H "Content-Type: application/json" \
-X POST
"https://networksecurity.googleapis.com/v1alpha1/projects/${PROJECT_ID}/locations/${REGION}/a
uthzPolicies?authz_policy_id=agent-gateway-ma-policy" \
-d '{
  "name": "agent-gateway-ma-policy",
  "policyProfile": "CONTENT_AUTHZ",
  "action": "CUSTOM",
  "target": {
    "resources": [
      "projects/"'"${PROJECT_ID}"'"/locations/"'"${REGION}"'"/agentGateways/agent-gateway"
    ]
  },
  "customProvider": {
    "authzExtension": {
      "resources": [
        "projects/"'"${PROJECT_ID}"'"/locations/"'"${REGION}"'"/authzExtensions/agent-gateway-
ma-authz"
      ]
    }
  }
}'
```

自訂 DLP 範本

Model Armor `sdpSettings.basicConfig` 使用內建的資訊類型清單。如要進行更精細的控制 (自訂資訊類型、部分遮蓋、代理值取代、依可能性遮蓋)，請透過 `sdpSettings.advancedConfig` 將 Model Armor 指向您自己的 Cloud DLP 檢查和去識別化範本。

建立檢查範本，標記 `POSSIBLE` 可能性或以上的美國社會安全號碼：

```
curl -fsS -X POST \
-H "Authorization: Bearer $(gcloud auth print-access-token)" \
-H "Content-Type: application/json" \
-H "x-goog-user-project: ${PROJECT_ID}" \
"https://dlp.googleapis.com/v2/projects/${PROJECT_ID}/locations/${REGION}/inspectTemplates" \
-d '{
  "templateId": "agw-ssn-inspect-template",
  "inspectTemplate": {
    "displayName": "SSN Inspect Template",
    "inspectConfig": {
      "infoTypes": [
        { "name": "US_SOCIAL_SECURITY_NUMBER" }
      ],
      "minLikelihood": "POSSIBLE"
    }
  }
}'
```

建立去識別化範本，將每個發現項目替換為資訊類型權杖 (例如 [US_SOCIAL_SECURITY_NUMBER])：

```
curl -fsS -X POST \
  -H "Authorization: Bearer $(gcloud auth print-access-token)" \
  -H "Content-Type: application/json" \
  -H "x-goog-user-project: ${PROJECT_ID}" \

"https://dlp.googleapis.com/v2/projects/${PROJECT_ID}/locations/${REGION}/deidentifyTemplates" \
-d '{
  "templateId": "agw-ssn-redaction-template",
  "deidentifyTemplate": {
    "displayName": "SSN Redaction Template",
    "deidentifyConfig": {
      "infoTypeTransformations": {
        "transformations": [{
          "primitiveTransformation": { "replaceWithInfoTypeConfig": {} }
        }]
      }
    }
  }
}
```

然後透過 `sdpSettings.advancedConfig` 將 Model Armor 範本的回應設定指向該配對 (如果您已連線，Terraform 的 `model_armor` 模組會在此設定 `advanced_config`)：

```
{
  "filterConfig": {
    "sdpSettings": {
      "advancedConfig": {
        "inspectTemplate":
"projects/${PROJECT_ID}/locations/${REGION}/inspectTemplates/agw-ssn-inspect-template",
        "deidentifyTemplate":
"projects/${PROJECT_ID}/locations/${REGION}/deidentifyTemplates/agw-ssn-redaction-template"
      }
    }
  }
}
```

上述 Model Armor CONTENT_AUTHZ 擴充功能會在下次評估回應時，自動採用這些進階設定範本，因此不需要變更擴充功能。確認 Model Armor 服務代理 (`service-${PROJECT_NUMBER}@gcp-sa-modelarmor.iam.gserviceaccount.com`) 在專案中持有 `roles/dlp.user` 和 `roles/dlp.reader` (Terraform 的 `model-armor` 模組已授予這些項目)。

IAP 輸出者 IAM (僅限每個 MCP 伺服器)

Terraform 不會在隱含的 IAP 代理程式登錄中建立專案範圍的 `roles/iap.egressor` 繫結。IAP REQUEST_AUTHZ 實際評估的繫結是每個 MCP 伺服器和每個推理引擎，在代理程式部署後且您知道代理程式 ID 時授予。「授予每個 MCP 伺

服务器的代理輸出權限」步驟會執行 `scripts/grant_agent_mcp_egress.sh` 。

11. 在 Cloud Run 中建構及部署 MCP 伺服器

`cloudrun/*.yaml.tpl` 和 `scaffold.yaml.tpl` 檔案會參照 `${PROJECT_ID}`、`${REGION}` 和 `${MCP_INGRESS}` (Cloud Run 傳入註解)。從 Terraform 輸出內容取得來源 `MCP_INGRESS`，確保轉譯的資訊清單與 `enable_cloud_run_private_networking` 同步，然後使用 `envsubst` 轉譯：

匯出 Cloud Run 輸入設定。

- `all`
- `internal-and-cloud-load-balancing` (使用私人網路方法時)

```
export MCP_INGRESS=all
```

返回根示範目錄：

```
cd ..
```

代入範本值：

```
envsubst '${PROJECT_ID} ${REGION} ${MCP_INGRESS}' < scaffold.yaml.tpl > scaffold.yaml
for f in cloudrun/*.yaml.tpl; do
  envsubst '${PROJECT_ID} ${REGION} ${MCP_INGRESS}' < "$f" > "${f%.tpl}"
done
```

每個 Cloud Run 服務都會以 Terraform 建立的服務專屬執行階段服務帳戶 (例如 `mcp-legacy-dms@${PROJECT_ID}.iam.gserviceaccount.com`) 執行。如要以這些服務帳戶部署，您必須擁有 `roles/iam.serviceAccountUser`：

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="user:$(gcloud config get-value account)" \
  --role="roles/iam.serviceAccountUser"
```

使用 Cloud Build 建構，並透過 Skaffold 部署：

```
skaffold run
```

Skaffold 會將三個映像檔 (`legacy-dms`、`corporate-email`、`income-verification-api`) 建構到 Artifact Registry 存放區，並更新每個 Cloud Run 服務，指向新的摘要。

驗證：

```
gcloud run services list --region=${REGION}
```

畫面上應會顯示所有三項服務，且狀態為 `ACTIVE`。

12. 將房貸代理部署至 Agent Runtime

安裝代理程式的依附元件並部署：

```
cd src/mortgage-agent
uv sync

uv run python deploy_agent.py \
  --project=${PROJECT_ID} \
  --region=${REGION} \
  --enable-agent-identity \
  --agent-name=mortgage-agent \
  --agent-gateway=projects/${PROJECT_ID}/locations/${REGION}/agentGateways/agent-gateway \
  --mcp-invoker-sa=$(terraform -chdir=../../terraform output -raw agent_mcp_invoker_email) \
  --model-endpoint-location=global
```

代理會在執行階段列出 `projects/${PROJECT_ID}/locations/${REGION}` 下的 Agent Registry 中的 `mcpServers`，探索 **MCP 工具**。

指令碼完成後，請將列印的 `reasoningEngines/` 複製到殼層 (例如 4262292559201566720)：

```
export AGENT_ID=<numeric-id-from-output>
cd ../../
```

13. 授予代理每個 MCP 伺服器的輸出權限

IAP REQUEST_AUTHZ 擴充功能會檢查代理的 `roles/iap.egressor`，授權每次工具呼叫特定 MCP 伺服器或端點。請參閱「[建立從代理程式到 MCP 伺服器的輸出政策](#)」。

指令碼 (`scripts/grant_agent_mcp_egress.sh`) 會列舉 `projects/${PROJECT_ID}/locations/${REGION}` 下代理程式登錄中的 MCP 伺服器，並將代理程式主體的 `roles/iap.egressor` 繫結合併至每個伺服器的 IAM 政策 (反映 `gcloud add-iam-policy-binding` 語意)。

用途 1：無條件授予特定 MCP 伺服器的範圍

```
./scripts/grant_agent_mcp_egress.sh \  
--mcp \  
--agent-id ${AGENT_ID} \  
--mcp-filter "legacy-dms income-verification"
```

應用情境 2：條件式授權 (CEL)，範圍限定於特定 MCP 伺服器

若要限制代理只能使用單一 MCP 伺服器上的一組工具，請附加 IAM 條件。Agent Gateway 會發布 IAP REQUEST_AUTHZ 向 CEL 公開的工具屬性，包括：

- `iap.googleapis.com/mcp.toolName`
- `iap.googleapis.com/mcp.tool.isReadOnly`
- `iap.googleapis.com/request.auth.type`。

將代理程式限制為只能使用 唯讀工具：`corporate-email`

```
./scripts/grant_agent_mcp_egress.sh \  
--mcp \  
--agent-id ${AGENT_ID} \  
--mcp-filter "corporate-email" \  
--condition-expression "api.getAttribute('iap.googleapis.com/mcp.tool.isReadOnly', false) == true" \  
--condition-title "ReadOnlyToolsOnly" \  
--condition-description "Restrict ${AGENT_ID} to read-only tools on corporate-email"
```

執行這項作業後，`corporate-email` 上的寫入工具會從 IAP REQUEST_AUTHZ 傳回 `403 PermissionDenied`，唯讀工具則會繼續運作。

驗證繫結

前往「政策」分頁，即可查看針對端點和 Mcp 伺服器建立的政策清單。

如果沒有任何端點違反政策，請使用下列設定執行指令碼。`./grant_agent_mcp_egress.sh --agent-id $AGENT_ID --endpoints`

其他用途：

每個 MCP 伺服器上無條件授予權限，範圍限定為一個代理

每次重新部署代理程式後，請執行這項指令。如果沒有篩選器和條件，具名代理程式會取得註冊資料庫中每個 MCP 伺服器的 `roles/iap.egressor`：

```
./scripts/grant_agent_mcp_egress.sh \  
  --mcp \  
  --agent-id ${AGENT_ID}
```

14. 在 Agent Platform 控制台測試代理程式

Agent Platform 控制台隨附 Playground，可讓您直接與已部署的代理程式對話。這是將代理程式連線至 Gemini Enterprise 前，快速進行工具呼叫的煙霧測試和檢查追蹤記錄的方法。

1. 在 Google Cloud 控制台中開啟「Agent Platform Deployments」(代理程式平台部署作業) 頁面。
2. 如要縮小執行階段清單範圍，請使用「篩選器」欄位，然後按一下 `mortgage-agent` 執行階段。
3. 開啟「Playground」分頁。
4. 輸入提示詞，與代理對話：

```
I am reviewing the Sterling familys current application. Can you summarize their 2024 and 2025 tax returns and verify if their total household income meets our 2026 debt-to-income requirements?
```

這項操作應會傳回文件管理工具和收入驗證工具的回應，且回應中應會遮蓋 SSN。

```
Can you send a summary of this to my email jane@example.com
```

由於 IAP 擴充功能處於模擬測試模式，因此系統不會強制執行條件式政策，代理程式將可順利傳送電子郵件。

由於代理程式是透過 OpenTelemetry 檢測功能部署，Playground 會顯示四個側邊面板檢視畫面，您可以在代理程式回覆時切換檢視畫面：

- **追蹤：**對話的完整追蹤記錄，包括 Agent Gateway、IAP REQUEST_AUTHZ 和 Model Armor CONTENT_AUTHZ 範圍
- **事件：**目前回合中叫用的工具和事件詳細資料圖表
- **狀態：**代理的工作階段狀態和工具輸入/輸出內容
- **工作階段：**您針對這個執行階段啟動的所有工作階段

Playground 完全略過 Gemini Enterprise 驗證，因此是確認 IAP REQUEST_AUTHZ 擴充功能是否正確授予工具呼叫權限的最快方式。如果工具在此處傳回 403，請先重新執行 `scripts/grant_agent_mcp_egress.sh`，再進行更深入的偵錯。

15. 強制執行 IAP 授權

現在我們已驗證部署作業，可以將 IAP 強制執行模式更新為 `null`，強制執行政策。開啟 `terraform.tfvars`，並將模式從 `DRY_RUN` 更新為 `null`

```
# IAP Enforcement Mode ("DRY_RUN" or null)
agent_gateway_iap_iam_enforcement_mode = null
```

套用變更。

```
terraform apply
```

返回 Playground，然後再次嘗試對話。

1. 在 Google Cloud 控制台中開啟「Agent Platform Deployments」(代理程式平台部署作業) 頁面。
2. 如要縮小執行階段清單範圍，請使用「篩選器」欄位，然後按一下 `mortgage-agent` 執行階段。
3. 開啟「Playground」分頁。
4. 輸入提示詞，與代理對話：

```
I am reviewing the Sterling familys current application. Can you summarize their 2024 and 2025 tax returns and verify if their total household income meets our 2026 debt-to-income requirements?
```

這項操作應會傳回文件管理工具和收入驗證工具的回應，且回應中應會遮蓋 SSN。

```
Can you send a summary of this to my email jane@example.com
```

如果一切設定正確，服務專員應會回覆，由於授權政策的關係，無法傳送電子郵件。

步驟 16 · 約 15 分鐘

16. 設定及測試 Gemini Enterprise

設定 Gemini Enterprise

請參閱 [Gemini Enterprise 入門指南](#)。

向 Gemini Enterprise 註冊 ADK 代理

請按照步驟在 Gemini Enterprise 中註冊代理程式，詳情請參閱[這篇文章](#)。

1. 前往 Google Cloud 控制台的「Gemini Enterprise」頁面。
2. 選取註冊代理的 Gemini Enterprise 應用程式。
3. 開啟「Your Gemini Enterprise webapp is ready」(Gemini Enterprise 網頁應用程式已準備就緒) 部分中顯示的網址。
4. 選取左選單中的「代理」分頁標籤，開啟[代理庫](#)。
5. 選取「Mortgage Assistant Agent」，然後開始對話。

在 Agent Runtime Playground 中嘗試相同的提示：

初始提示：

```
I am reviewing the Sterling familys current application. Can you summarize their 2024 and 2025 tax returns and verify if their total household income meets our 2026 debt-to-income requirements?
```

後續查詢提示：

```
Can you send a summary of this to my email jane@example.com
```

返回控制台的「Agent Deployment」部分，選取代理部署作業並前往「traces」分頁，您現在會在顯示 Gemini Enterprise 發起呼叫的範圍中，看到 Gemini Assistant 代理。

17. 疑難排解與常見修正方式

- `terraform apply` 在代理程式閘道上失敗，並顯示「**resource is being created and therefore can not be updated**」(正在建立資源，因此無法更新)，這是因為閘道的租戶專案需要約 30 秒才能完成設定，之後才能附加授權政策。模組的 `time_sleep.wait_for_gateway` 會處理這項作業，只要重新執行 `terraform apply` 即可。
- 代理回報「找不到 MCP 伺服器」或僅使用公用程式工具啟動 - 確認 `enable_agent_registry_endpoints = true` 位於 `terraform.tfvars`，然後：

```
gcloud alpha agent-registry mcp-servers list \
  --project=${PROJECT_ID} --location=${REGION}
```

您應該會看到三個項目 (每個 Cloud Run MCP 服務各一個)。如果清單空白，請確認 MCP 服務可從虛擬私有雲內部連線，且 Agent Gateway 已填入登錄檔 (第一次代理工具清單時，系統會延遲執行這項操作)。

- 工具呼叫傳回 **403 PermissionDenied** - 重新執行 `scripts/grant_agent_mcp_egress.sh`。最常見的原因是忘記在重新部署代理程式後重新授予權限 (每次部署時 `reasoningEngines/` 都會變更)。
 - `skaffold run` 失敗並顯示「服務帳戶權限遭拒」，表示您缺少 `roles/iam.serviceAccountUser`。重新執行上一步的自我授權。
 - 從 Agent Gateway 到 MCP LB 的 DNS 對等互連錯誤：確認 `agent_gateway_dns_peering_config.target_network` 與 `projects/${PROJECT_ID}/global/networks/${VPC_NAME}` 完全相符，且每個 `domains` 項目都以尾隨點結尾。
 - `terraform plan` 一直想更新 Cloud Run 映像檔標記，但由於 `lifecycle { ignore_changes }` 規則，這不應該發生。如果確實如此，請確認您未在 `skaffold run` 之後編輯 `terraform.tfvars` 中的 `mcp_services[*].image`。
-

步驟 18 · 約 10 分鐘

18. 清除所用資源

推論引擎「不」由 Terraform 管理 (ADK SDK 會建立推論引擎)。手動刪除：

```
gcloud beta ai reasoning-engines delete ${AGENT_ID} \  
  --region=${REGION} --project=${PROJECT_ID}
```

拆除 Terraform 建立的所有項目：

```
cd terraform  
terraform destroy  
cd ..
```

如果您建立公開 DNS 區域只是為了完成這個程式碼研究室：

```
gcloud dns managed-zones delete agw-example-com
```

最後，請刪除 Terraform 狀態值區：

```
gcloud storage rm -r gs://${PROJECT_ID}-tfstate
```

19. 恭喜

恭喜！您已成功使用 Agent Gateway，為多工具 ADK 代理導入完整的代理控管機制。Agent Gateway 可做為集中式網路控制層，讓您建立通往私人工具的安全輸出路徑、透過 Identity-Aware Proxy 強制執行以身分為準的精細 IAM 政策，以及使用整合式 Model Armor 防護措施清除內容互動。

目前所學內容

- 如何部署及設定代理閘道，做為 Agent-to-Anywhere 輸出流量的中央管理層。
- 如何整合 Agent Registry，以探索受控的動態執行階段工具。
- 如何編寫及強制執行以工具和條件為依據的 IAM 政策，嚴格控管代理程式執行路徑。
- 如何運用 Agent Gateway 服務擴充功能套用 Model Armor 政策，自動攔截及修訂機密代理程式流量。

參考文件

- [代理程式管理政策總覽](#)
 - [Agent Gateway 說明文件](#)
 - [代理程式身分說明文件](#)
 - [Gemini Enterprise Agent Platform 總覽](#)
-